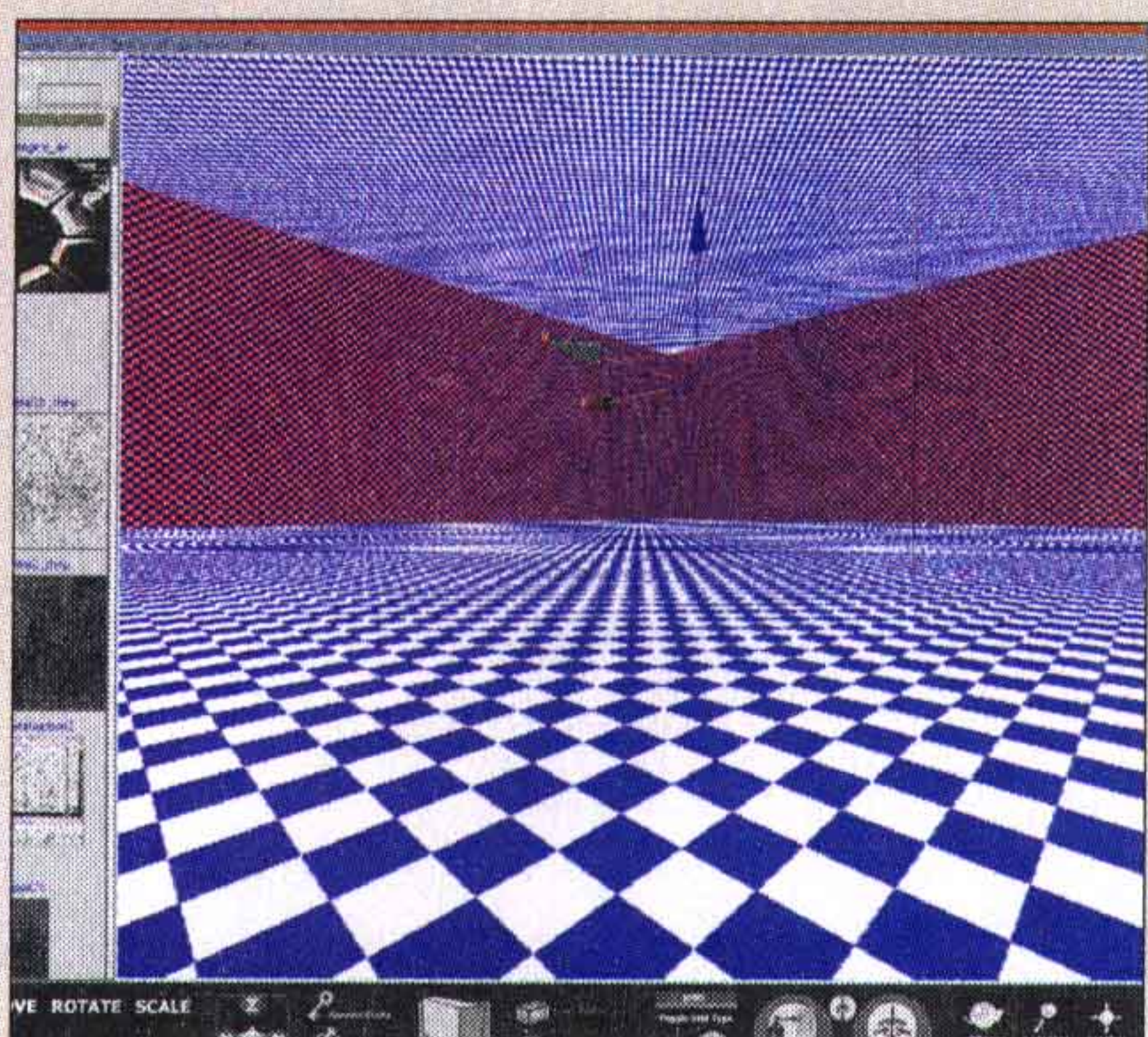




The walls, floor and ceiling of the level are selected to apply a texture



Repeat these steps for the remaining walls, floor and ceiling. A ceiling can either have a texture similar to a wall or a sky shader like storm clouds, depending upon the look that you wish to impart to the game.



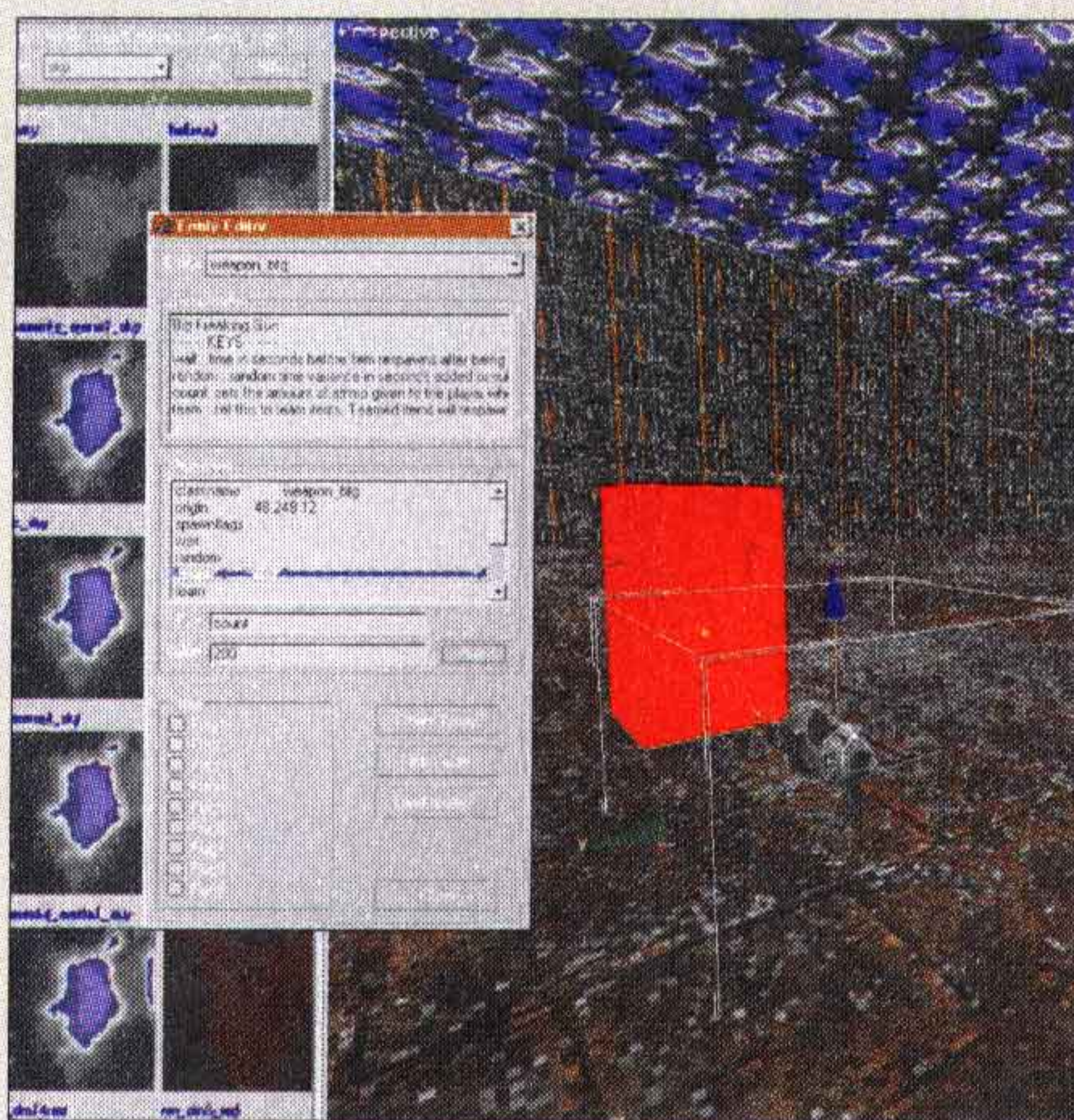
Next, we need to add a player start entity to the map, without which the level will not load properly into *Quake III Arena*. Activate the snaps by using the Snap button. Right-click in the viewport to will bring up the quad menu. Select AutoPosition Grid Upper to move the Snap the Height Grid tool to the same base level as the floor.



Click the #8 Grid Spacing button to set the snap grid level. Click on the PlayerStart button in the toolbar. Click anywhere on the floor to create a PlayerStart entity—it will appear as a rectangular box.



To add a weapon, we need to create a Generic Entity. Click on the appropriate button in the Flash toolbar and click anywhere on the floor to place the entity. Click on the Entity Editor button located below the Shader Nav button from the drop down list, select weapon_bfg to transform your generic entity into a BFG (Big F***** Gun).



A weapon is nothing but a Generic Entity with specific properties



To set the ammunition for the BFG, head to Properties in the Entity Editor and select the count value. Click on the Value Field and type 200. Click Set to allocate 200 rounds to the BFG.



Save your map as a .gmax file and get ready to test your level. Click on the Compile button in the lower left of the toolbar. Click next on Normal Compile to create a .map file of your level—the compiler will convert this into a .bsp file.



Now, fire up your copy of *Quake III Arena* to load your new level: Hit [~] to access the console, type in `sv_pure 0` and then `map (name of the map you just compiled)`. Now play your level!



Your level is now ready for testing

a track with elements of sharp and gentle curves, road elevations, hazards, blind corners, shortcuts and multiple-routes.

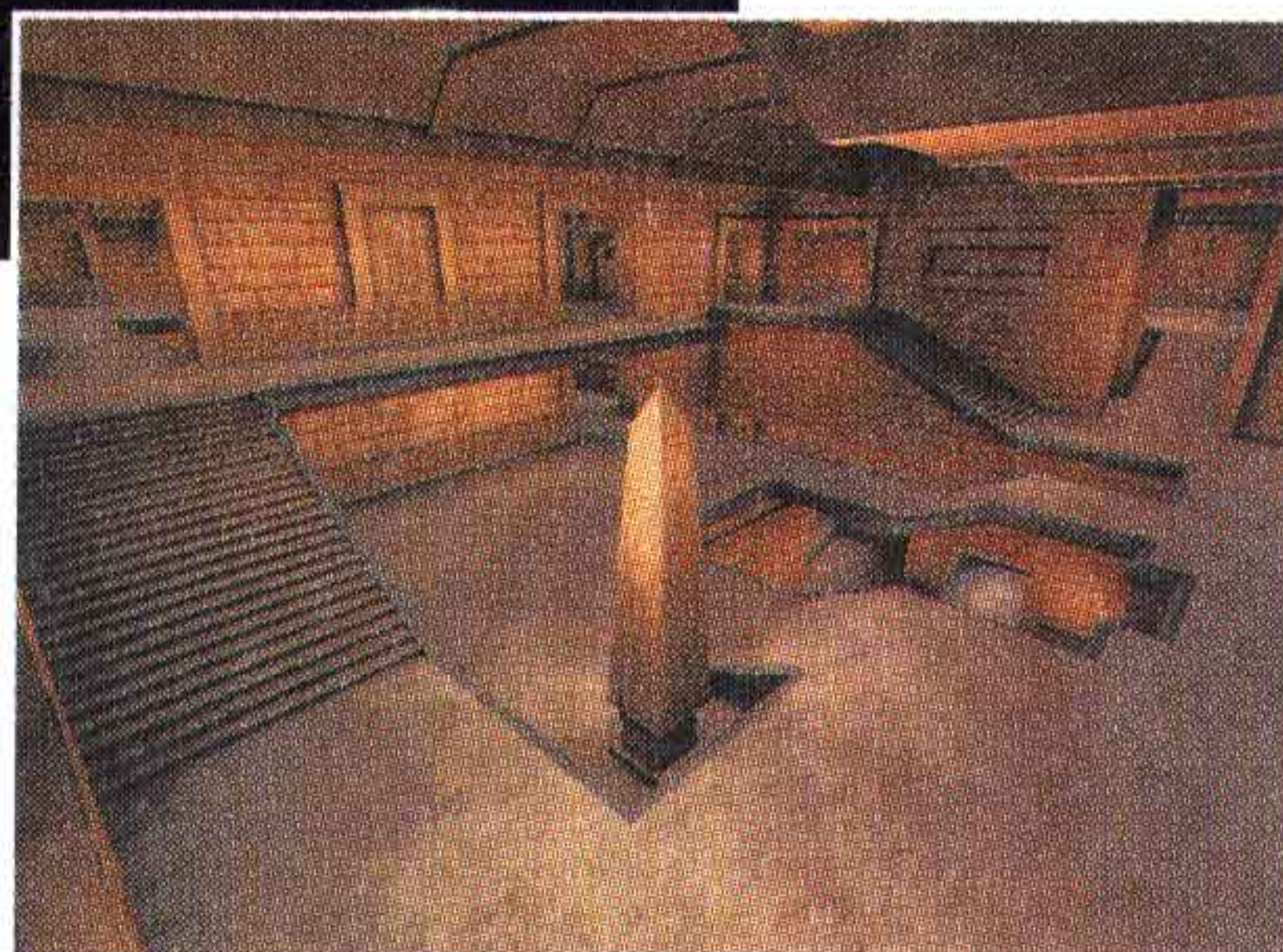
■ While a racing game is reflexive, a real-time strategy is a more laid-back, mental exercise. Every level has to ensure that the most important element of this genre, the resources, is well represented. Whether wood, water or gold, a resource needs to be placed symmetrically, offering a level playing field to the parties playing. Other elements that require intelligent placement are ground elevations, choke points and control points.

■ In role-playing games, resources take on the form of monsters, non-playing characters (NPCs), weapons and magical items. These should be placed between encounters with baddies and conversations with friends in a manner that ensures continuity in the gameplay and a steadily rising learning and experience curve.

■ Even in an action game, the levels steal the show from the weapons and define



Good multiplayer maps have two elements in common—multiple access points and a symmetrical layout



the best or the worst moments that a player takes away from the game. A single-player FPS romp is based on the dual principles of suspense and isolation. A level should thus possess a maze-like

quality; encounters with the enemy must be balanced such that a player makes it through the situation, but barely. Multiplayer versions of this genre need a significantly different level. While a solitary stroll needs a long, linear environment, strewn with puzzles and encounters; multiplayer frag fests need short, almost circular levels, complete with symmetric weapon placements and entry and exit points.

These are of course generalisations on what makes a good level. Observe the works of others, practise and try to innovate—spice up that level with hidden rooms and Easter eggs, mechanical traps and unique puzzles. Above all, have fun. After all, that's the name of the game, right?

AHMED SHAIKH